Assignment Number One

Layyana Junaid (23k0056)

Alisha Zaidi (23k0025)

1. Introduction

The assignment was to build, train, inspect and evaluate a small causal language model with our own Transformer implementation, without loading any pretrained model. The scenario is a small applied-AI team that needs to know whether a compact Transformer trained from scratch can learn the next-character structure of a narrow domain well enough to generate coherent text. We used Tiny Shakespeare as the domain corpus.

Our pipeline is: raw text, then a character tokenizer, then fixed-length windows with shifted targets, then token embedding plus sinusoidal positions, then four causal Transformer blocks, then a final LayerNorm and a vocabulary projection, then sampled text. The model is available as a command-line tool and as a FastAPI service deployed on Render.

We judged the work against four questions. Is the attention mechanism correct, shown by a hand calculation and unit tests? Does training work, shown by an overfit gate and loss curves? Are the conclusions backed by controlled experiments? And can someone else reproduce the results from the repository?

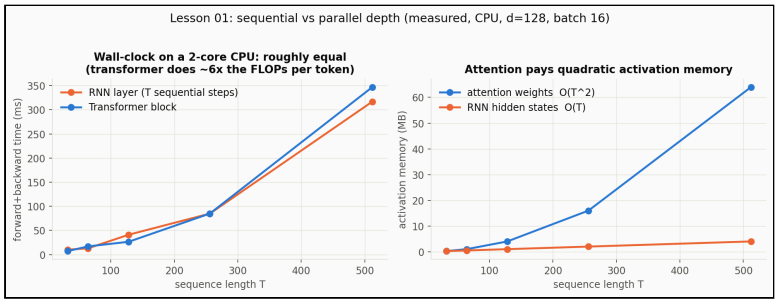
2. Transformer Foundations

2.1 Sequential and parallel depth

A recurrent layer computes h_t = tanh(W x_t + U h_{t-1}). Step t cannot start until step t-1 has finished, so a sequence of length T needs T dependent steps, and the path between the first and last token is T steps long. Self-attention connects every pair of positions in one operation. Each layer needs a fixed number of sequential matrix multiplications (six in our implementation), whatever the length. The price is memory and arithmetic: attention stores a T by T weight matrix per head and costs $O(T^2 d)$, while a recurrent layer costs $O(T d^2)$ and stores $O(T d)$.

We timed one RNN layer against one Transformer block (d = 128, batch 16, forward and backward, best of three runs, 2-core CPU) to check the trade-off (`experiments/seq_vs_parallel.py`).

T	RNN layer (ms)	Transformer block (ms)	RNN sequential steps	Attention weights (MB)	RNN hidden states (MB)
32	10.1	7.1	32	0.25	0.25
128	41.0	26.4	128	4.0	1.0
256	84.8	84.9	256	16.0	2.0
512	317.0	347.1	512	64.0	4.0



On this CPU the wall-clock times are similar, and the Transformer block is faster at T = 128 but slower at T = 512. Attention memory grows quadratically, from 0.25 MB at T = 32 to 64 MB at T = 512. Our interpretation is that two cores cannot exploit the constant sequential depth, so the main advantage of attention does not appear in these

timings. Per token, a Transformer block does about $12d^2$ multiply-adds (four projections and a 4× feed-forward layer) against about $2d^2$ for an RNN cell. That is roughly six times the arithmetic in a similar time. This comes from counting operations, not from a separate measurement. On a GPU the sequential-depth difference would matter much more.

2.2 Scaled dot-product attention and the causal mask

For queries Q, keys K and values V, each of shape (T, d_k):

```
scores  = Q Kᵀ / √d_k
weights = softmax(scores + mask)      (row-wise)
output  = weights V
```

Dividing by √d_k keeps the scores at a stable scale. If q and k have unit-variance components, q·k has variance d_k, and large scores push the softmax into a nearly one-hot regime where gradients vanish. The test `test_scaling_by_sqrt_dk_matters` checks this.

We first worked a 3-token example with d_k = d_v = 2 by hand. The inputs are Q = K = [[1,0],[0,1],[1,1]] and V = [[1,0],[0,2],[3,1]].

Step	**Result**
$Q K^T$	[[1, 0, 1], [0, 1, 1], [1, 1, 2]]
$Q K^T / \sqrt{2}$	[[0.707, 0, 0.707], [0, 0.707, 0.707], [0.707, 0.707, 1.414]]
Row-wise softmax	[[0.4011, 0.1978, 0.4011], [0.1978, 0.4011, 0.4011], [0.2483, 0.2483, 0.5035]]
Weights × V	[[1.6044, 0.7967], [1.4011, 1.2033], [1.7587, 1.0000]]
Causal mask, then softmax	[[1, 0, 0], [0.3302, 0.6698, 0], [0.2483, 0.2483, 0.5035]]
Masked output	[[1, 0], [0.3302, 1.3395], [1.7587, 1.0]]

The script `experiments/hand_calculation.py` prints every step and asserts that the library function gives the same values (Figure 1).

```
HAND CALCULATION: scaled dot-product attention, 3 tokens, d_k = d_v = 2

1. Matrices (rows = tokens 1..3)
  Q =
    [      1        0 ]
    [      0        1 ]
    [      1        1 ]
  K =
    [      1        0 ]
    [      0        1 ]
    [      1        1 ]
  V =
    [      1        0 ]
    [      0        2 ]
    [      3        1 ]

2. Q K^T  (entry [i,j] = q_i . k_j)
    [      1        0        1 ]
    [      0        1        1 ]
    [      1        1        2 ]
   e.g. [3,3] = (1)(1) + (1)(1) = 2 ;  [1,2] = (1)(0) + (0)(1) = 0

3. Divide by sqrt(d_k) = sqrt(2) = 1.4142
    [  0.707    0.000    0.707 ]
    [  0.000    0.707    0.707 ]
    [  0.707    0.707    1.414 ]

4. Row-wise softmax  (each row sums to 1)
    [ 0.4011   0.1978   0.4011 ]
    [ 0.1978   0.4011   0.4011 ]
    [ 0.2483   0.2483   0.5035 ]
   row 1: exp(.707)=2.028, exp(0)=1, exp(.707)=2.028 -> sum 5.056 -> [0.4011, 0.1978, 0.4011]

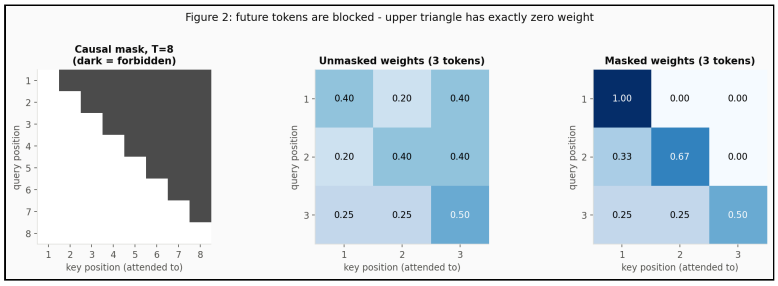
5. Output = weights @ V  (each row = weighted average of the value vectors)
    [ 1.6044   0.7967 ]
    [ 1.4011   1.2033 ]
    [ 1.7587   1.0000 ]
   row 1: 0.4011*[1,0] + 0.1978*[0,2] + 0.4011*[3,1] = [1.6044, 0.7967]

6. Causal mask: add -inf above the diagonal (token 1 cannot see 2,3; token 2 cannot see 3)
  masked scores =
    [  0.707     -inf     -inf ]
    [  0.000    0.707     -inf ]
    [  0.707    0.707    1.414 ]
  masked softmax weights =
    [ 1.0000   0.0000   0.0000 ]
    [ 0.3302   0.6698   0.0000 ]
    [ 0.2483   0.2483   0.5035 ]
  masked output =
    [ 1.0000   0.0000 ]
    [ 0.3302   1.3395 ]
    [ 1.7587   1.0000 ]
   row 1 = [1, 0] = v_1 exactly: with only itself visible, token 1 copies its own value.
   row 2: scores [0, 0.707] -> weights [1/(1+2.028), 2.028/(1+2.028)] = [0.3302, 0.6698]
          output = 0.3302*[1,0] + 0.6698*[0,2] = [0.3302, 1.3395]
   row 3: identical to the unmasked row, because token 3 may already see everyone.

7. What changes: unmasked, every token mixes information from ALL positions (including the
   future). Masked, token i receives a convex combination of tokens 1..i only, so information
   flows strictly left-to-right. Row 1 loses all context; row 3 is unchanged.

VERIFIED: matches src.attention.scaled_dot_product_attention (masked and unmasked).
```

The causal mask sets every entry above the diagonal to $-\infty$ before the softmax. Since $\exp(-\infty) = 0$, future positions get exactly zero weight, and each row still sums to 1. If we had used 0 instead, the future would still receive a positive weight. Figure 2 shows the mask and its effect.



Without the mask, every token mixes information from all positions, including the future. With the mask, token i receives a weighted average of tokens 1 to i only. Token 1 can only copy its own value, so its output is exactly v_1. Token 3 can already see every position, so its row is unchanged. Information flows strictly left to right, which is what makes next-token training valid.

2.3 Multi-head attention

With H heads, each head has dimension d_k = d_model / H. The input x of shape (B, T, D) is projected by W_q, W_k and W_v, and each projection is split into heads with `view` and `transpose`, giving (B, H, T, d_k). Attention runs independently in each head. The heads are merged back to (B, T, D) by concatenation, and an output projection W_o is applied. For B = 2, T = 16, D = 128 and H = 4, the shapes are (2, 16, 128) for the input, (2, 4, 16, 32) per split tensor, (2, 4, 16, 16) for the attention weights, and (2, 16, 128) after the merge and W_o. Figure 3

is the printed trace from the code. The check `merge(split(x)) == x` returns True, and a unit test confirms that head 0 receives features 0 to 3.

```
Figure 3: tensor shapes through multi-head attention
MultiHeadAttention(d_model=128, n_heads=4) -> d_k = d_model / n_heads = 32
batch B=2, sequence T=16
----------------------------------------------------------------------
input x                          (2, 16, 128)
Q = x W_q                        (2, 16, 128)
Q split into heads               (2, 4, 16, 32)
K split into heads               (2, 4, 16, 32)
V split into heads               (2, 4, 16, 32)
attention weights (per head)     (2, 4, 16, 16)
per-head output = weights @ V    (2, 4, 16, 32)
heads concatenated               (2, 16, 128)
output projection W_o            (2, 16, 128)
----------------------------------------------------------------------
split : (B,T,D) -> view (B,T,H,d_k) -> transpose -> (B,H,T,d_k)
merge : (B,H,T,d_k) -> transpose -> (B,T,H,d_k) -> view (B,T,D)   [= concatenation]
check : merge(split(x)) == x  ->  True
```

2.4 Positional encoding

Attention is permutation-equivariant, so the model needs position information. We add a fixed sinusoidal code to each token embedding: PE[pos, 2i] = sin(pos / 10000^(2i/d)) and PE[pos, 2i+1] = cos(pos / 10000^(2i/d)).

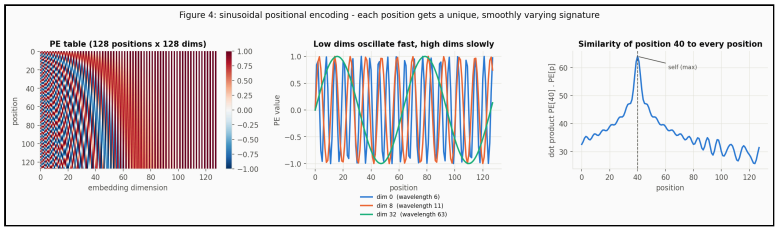


Figure 4 shows the 128 × 128 table, the frequency of each dimension, and the dot product between one position and all others. We also checked that position matters. When the same token embedding is placed at positions 0 to 5, the pairwise L2 distances after adding the encoding are 1.95 for neighbours and 3.64, 4.86, 5.55 and 5.80 for larger gaps. Identical tokens can be told apart only by position, and the distance grows with the gap.

2.5 The Transformer block and the full model

Each block uses pre-LayerNorm, as in GPT-2:

```
x = x + Dropout(MHA(LN₁(x)))        attention with causal mask, then residual
x = x + Dropout(FFN(LN₂(x)))        FFN = Linear(d → 4d) → GELU → Linear(4d → d), then
residual
```

The model is the token embedding (scaled by √d_model) plus the positional encoding, then N blocks, then a final LayerNorm and a linear head to 66 logits. Figure B shows the architecture and Figure 5 the printed summary from the trained checkpoint.

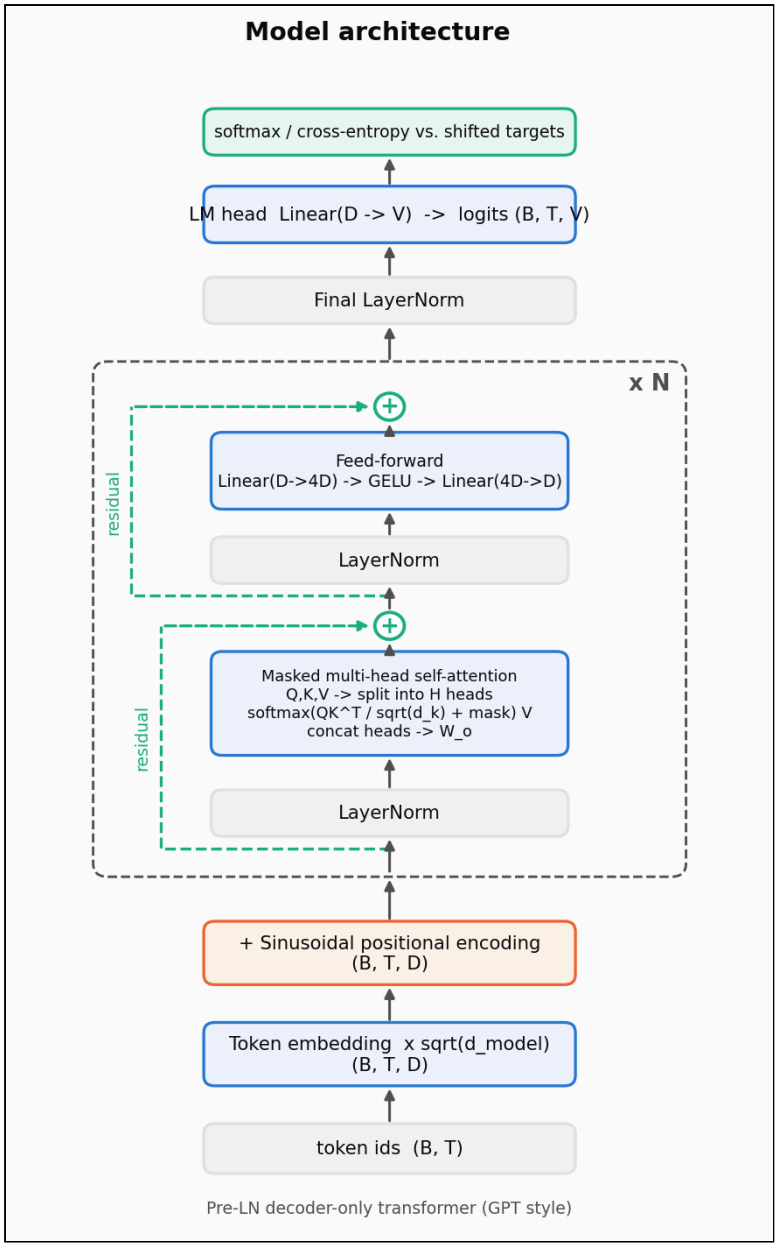

Model architecture
softmax / cross-entropy vs. shifted targets
LM head Linear(D -> V) -> logits (B, T, V)
Final LayerNorm
x N
residual
Feed-forward
Linear(D->4D) -> GELU -> Linear(4D->D)
LayerNorm
residual
Masked multi-head self-attention
Q,K,V -> split into H heads
softmax(QK^T / sqrt(d_k) + mask) V
concat heads -> W_o
LayerNorm
+ Sinusoidal positional encoding
(B, T, D)
Token embedding x sqrt(d_model)
(B, T, D)
token ids (B, T)
Pre-LN decoder-only transformer (GPT style)

Figure 5: model summary / parameter count

```
MiniGPT summary  [trained checkpoint minigpt.pt (val loss 1.6470)]
============================================================
configuration:
  vocab_size       66
  context_length   128
  d_model          128
  n_heads          4
  n_layers         4
  ffn_dim          512
  dropout          0.1
  pos_encoding     sinusoidal
  tie_weights      False
  causal           True
------------------------------------------------------------
component                          shape / detail       params
------------------------------------------------------------
token embedding                    (66, 128)             8,448
positional encoding (sinusoidal)   fixed buffer              0
block 0: attention (Wq,Wk,Wv,Wo)   4 heads x 32          66,048
block 0: FFN (GELU)                128->512->128        131,712
block 0: 2 x LayerNorm                                      512
block 1: attention (Wq,Wk,Wv,Wo)   4 heads x 32          66,048
block 1: FFN (GELU)                128->512->128        131,712
block 1: 2 x LayerNorm                                      512
block 2: attention (Wq,Wk,Wv,Wo)   4 heads x 32          66,048
block 2: FFN (GELU)                128->512->128        131,712
block 2: 2 x LayerNorm                                      512
block 3: attention (Wq,Wk,Wv,Wo)   4 heads x 32          66,048
block 3: FFN (GELU)                128->512->128        131,712
block 3: 2 x LayerNorm                                      512
final LayerNorm                                             256
LM head (untied)                   (128, 66)             8,448
------------------------------------------------------------
TOTAL trainable parameters                              810,240
  (0.810 M; sum of rows = 810,240; fp32 size = 3.1 MiB)
```

Each block has $12d^2 + 13d = 198{,}272$ parameters at $d = 128$. The total is:

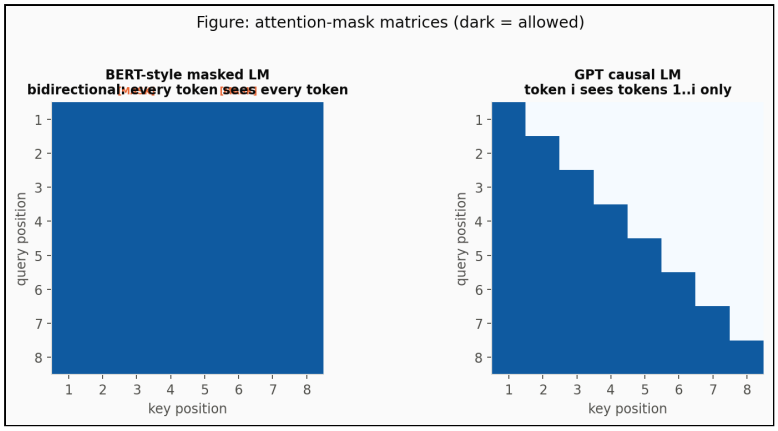
Part	**Parameters**
Token embedding (66 × 128)	8,448
Four blocks (4 × 198,272)	793,088
Final LayerNorm	256
LM head (128 × 66, no bias)	8,448
Total	**810,240**

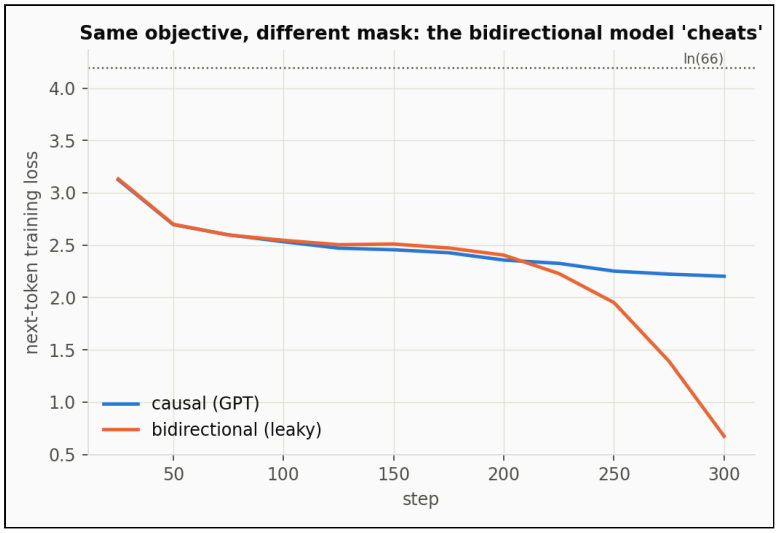
A unit test checks the total against the model.

2.6 Causal and masked language modelling

GPT factorises the probability of a sequence as a product of next-token probabilities, so it needs a causal mask. BERT predicts masked tokens from context on both sides, so it uses a full mask. A bidirectional model can be trained to predict tokens it can see, which is a leak. We tested this with the same 2-layer model ($d = 64$, 300 steps, same data and seed), trained once with a causal mask and once with a full mask.

Mask used in training	**Final train loss**	**Validation loss (mask as trained)**	**Validation loss with causal mask**
Causal (GPT)	2.203	2.255	2.255
Full (bidirectional)	0.674	0.678	5.692





The two losses are nearly identical until about step 200. After that the bidirectional model's loss drops sharply. Evaluated with the causal mask that generation uses, it scores 5.69, which is worse than the uniform guess of 4.19. Our interpretation is that a full mask lets position t see token t+1, which is its target, so the model learns to copy the answer. The low loss is a leak and does not reflect language modelling. The tests `test_full_model_is_causal` and `test_bidirectional_model_is_NOT_causal` check the causal property directly.

2.7 The next-token objective

For each window of T+1 tokens, the input is w[:-1] and the target is w[1:], so the target at position t is the token after input t. The loss is cross-entropy over the 66 characters at every position. At initialisation the model should be uniform, which gives ln 66 = 4.19. The untrained model scores 4.22, so the initialisation is close to neutral.

3. Implementation

Component	File	Notes
Tokenizer	`src/tokenizer.py`	Character level. Vocabulary is the sorted set of characters plus `<unk>`, which gives 66 tokens. No BOS, EOS or PAD tokens are needed.
Dataset	`src/dataset.py`	Random windows of 129 characters, split into inputs and shifted targets.
Attention	`src/attention.py`	Scaled dot-product attention, causal mask, and multi-head attention with explicit split and merge.
Positions	`src/positional_encoding.py`	Sinusoidal (used in the final model) and learned (ablation only).
Block and model	`src/transformer_block.py`, `src/gpt.py`	Pre-LN block, GELU feed-forward, residuals, final LayerNorm, LM head.
Training	`src/train.py`	AdamW, warm-up then cosine decay, gradient clipping, fixed validation batches, JSON logs, best checkpoint.
Generation	`src/generate.py`	Autoregressive sampling over a sliding window of 128 characters, with greedy, temperature, top-k and top-p options.
Evaluation	`src/evaluate.py`	Full-split loss and perplexity, and five fixed-prompt samples.
Serving	`src/api.py`	FastAPI service with `/health`, `/generate` and `/docs`.

We made four design choices. Pre-LN keeps an unnormalised identity path through the residual stream, which trains stably without a long warm-up. Token embeddings are scaled by $\sqrt{d_model}$, because otherwise the positional code (values of magnitude about 1) would dominate the token identity. The output projection of each residual branch is scaled by $1/\sqrt{(2N)}$, so the residual variance does not grow with depth. Weight decay is applied to weight matrices only.

The suite has 36 tests in seven files. They check attention against the hand calculation, zero weight on future positions, causality of the whole model, shapes, positional encoding, the parameter count, the target shift, the overfit gate, checkpoint reload in a fresh process, and the CLI output. The test suite uses `F.scaled_dot_product_attention` as an independent reference, but the model does not call it. We also kept our earlier NumPy implementation with hand-derived backward passes (`reference/numpy_llm/`), and a test checks its gradients against PyTorch autograd.

4. Dataset and Training Setup

We used Tiny Shakespeare from Hugging Face (`karpathy/tiny_shakespeare`), which the script downloads automatically. It contains 1,115,394 characters and 65 distinct characters. The last 10% is the validation set, taken contiguously and not shuffled, which gives 1,003,855 training and 111,539 validation tokens. We applied no filtering. Because the split is contiguous, the validation text comes from different plays than most of the training text, so part of the train–validation gap reflects a distribution shift.

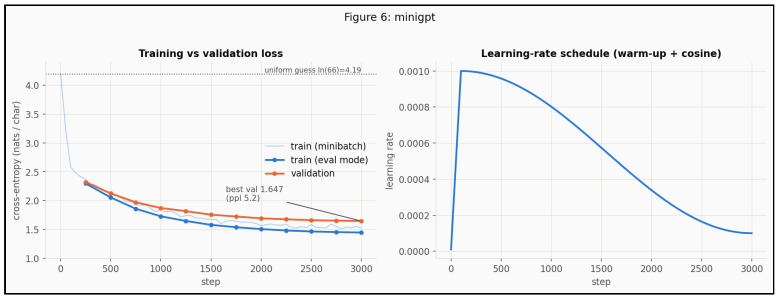
Item	Value
Hardware	2 vCPU cloud container, no GPU, peak process memory 1.44 GB
Model	4 layers, 4 heads, d_model 128, FFN 512, context 128, dropout 0.1, sinusoidal positions, 810,240 parameters
Optimiser	AdamW, β = (0.9, 0.95), weight decay 0.1 on matrices, gradient clipping 1.0
Learning rate	1e-3, 100 warm-up steps, cosine decay to 1e-4
Batch and steps	64 windows of 128 characters, 3,000 steps, about 24.5 passes over the training text
Wall-clock time	2,310 s (38.5 min), 10,637 tokens/s
Evaluation	Every 250 steps on 50 fixed-seed batches, dropout off; best validation checkpoint kept
Seed	1337

The handout suggests a learning rate of about 3e-4. We used 1e-3, which is common for small character-level models, and training was stable with no loss spikes. We did not run a learning-rate sweep, so this choice is untested.

Before the full run we ran the overfit gate: one fixed batch of 16 windows for 300 steps. The loss fell from 4.20 to 0.069 (`outputs/logs/overfit/summary.json`). This shows that gradients flow and that inputs and targets are aligned.

5. Results

5.1 Loss curves



Validation loss starts at the uniform baseline of 4.19 and ends at 1.647 at step 3,000. On the full validation split after training, the loss is 1.645, which is a perplexity of 5.18, or 2.37 bits per character. The final training loss is 1.450. Validation loss was 1.693 at step 2,000, so it was still falling at the end.

Our interpretation is that the gap of about 0.2 nats comes partly from the contiguous split and partly from limited capacity and training budget. We did not test this with a longer run, so it remains a reading of the curves.

5.2 Generated samples

We used five fixed prompts with temperature 0.8 and top-k 40. Each sample below is the model's output, unedited (`outputs/samples/eval_samples.txt`).

```
Prompt: ROMEO:
ROMEO:
If they world, but they to son.

CAMILLO:
As my lord, now legs in him: I fay, I see not
Reeson of the known that will name it, day your worse,
With is thou hast swear not intermits,
With on the crown by on a men against of see
And he opisale to it.

Prompt: To be, or not to be
To be, or not to be thee
With made ere him with an the could?

CLIFFORD:
Sunpar for Pardon, and desires, well:
Where were the combled friends: therefore:
hat, wheren fraid, are toward good him repour delier
And all to-moore my kindless well here these feels
At him pays

Prompt: KING HENRY:
KING HENRY:
Ah, you are by have speak come many good from thou have
In ever him to thirs in the grief be to being
Pempt was you bark, my countly give majesty,
Thou hast the wat we cault with us dear our forth.

ROMEO:
Sir, forth, and good-morrow, good sorrow pro

Prompt: First Citizen:
Before we proceed will go unto my lastifish,
Which gentlement for his both it the to cower:
Of cannot man's be and moder to me thee wast.

ROMEO:
See, my lord.

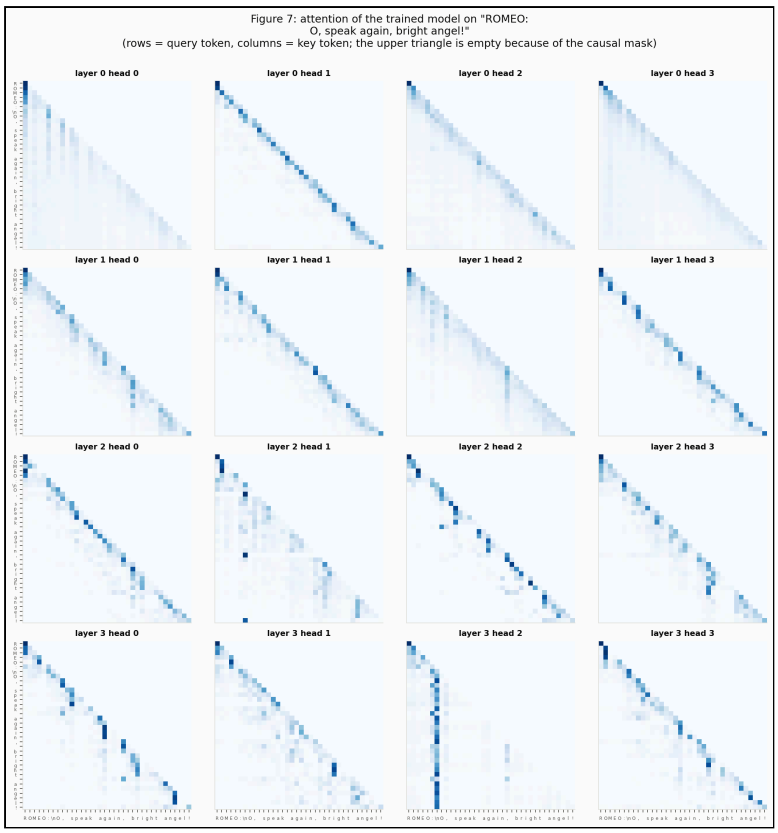
CAMILLO:
Here will the gonerning known provost,
And your both mindrel myself from fortences,
Thou lath the

Prompt: O Juliet,
O Juliet, poor Romeo and right
That setlems for my life, on Henry's with thee,
The work and ever not stand to the kingnow.

JULIET:
Why, have that as man with a world
That us a beather so hourts to me, sent the son,
I doo sue of Romeo, fool. We'll I have not t
```

The samples show the format of a play: speaker names in capitals, colons, line breaks, and short verse-like lines. Many words are real Elizabethan-style words. The model also invents words such as "Pempt", "cault" and "countly", and the speakers change without a clear reason.

5.3 Attention inspection



We averaged each head's attention over all query positions (`outputs/logs/attention_stats.txt`). The table shows three heads.

Layer, head	Mean weight on previous character	Mean weight on itself	Mean weight on first character
0, 1	0.503	0.177	0.072
2, 2	0.418	0.193	0.061
3, 2	0.081	0.090	0.080

Layer 0, head 1 places about half of its weight on the previous character, which means it works as a local filter. Layer 3, head 2 spreads its weight almost evenly and does not favour any of the three positions we measured. We did not establish what the deeper heads compute, so we make no claim about their roles. The upper triangle of every head is exactly zero, which confirms that the trained model respects the causal mask.

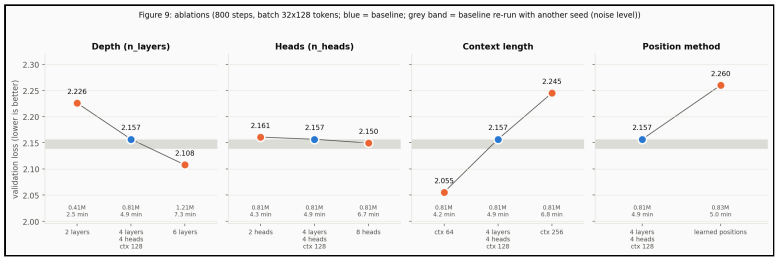
6. Ablation Study

6.1 Protocol

Each run changes one factor from the baseline (4 layers, 4 heads, context 128, sinusoidal positions) and keeps everything else fixed. Eight variants at 3,000 steps were too slow on a CPU, so each ablation runs for 800 steps at batch 32 × 128 characters (about 3.3 M characters). In the context experiment, the number of characters per step is held constant. To estimate noise, we re-ran the baseline with a second seed, which gave validation losses of 2.157 and 2.138. We therefore treat differences smaller than about 0.02 as noise.

6.2 Results

Run	Setting	Params	Val loss	Val perplexity	Train loss	Time (s)	Tokens/s
base	4 layers, 4 heads, context 128	810,240	2.157	8.64	2.105	297	11,042
base, seed 7	noise estimate	810,240	2.138	8.48	2.089	316	–
depth 2	2 layers	413,696	2.226	9.26	2.190	150	21,847
depth 6	6 layers	1,206,784	2.108	8.23	2.048	437	7,495
heads 2	2 heads	810,240	2.161	8.68	2.113	257	12,767
heads 8	8 heads	810,240	2.150	8.58	2.098	402	8,150
context 64	context 64	810,240	2.055	7.81	1.997	249	13,144
context 256	context 256	810,240	2.245	9.44	2.214	409	8,002
learned positions	learned table	826,624	2.260	9.58	2.228	301	10,878



The raw data is in `outputs/ablation/results.csv`. The table below separates what we measured from what we think it means.

Factor	Measured	Our interpretation
Depth, 2 → 4 → 6 layers	Validation loss 2.226 → 2.157 → 2.108. Time 150 → 297 → 437 s.	Depth helps, but each extra step gives less (−0.069, then −0.049) at higher cost. Depth is confounded with parameter count, since we did not run a width-matched control.
Heads, 2 / 4 / 8	2.161 / 2.157 / 2.150, all within noise. Time 257 → 297 → 402 s.	More heads gave no benefit at this size. The extra time probably comes from more, smaller matrix multiplies on CPU.
Context, 64 / 128 / 256	2.055 / 2.157 / 2.245. Throughput 13.1k → 11.0k → 8.0k tokens/s.	In this short budget the shortest context gave the lowest loss. Local character statistics may need little context, and the shorter-context batches contain more distinct windows. Neither explanation was tested, and the result does not show that long context is harmful.
Positions, sinusoidal vs. learned	2.157 vs. 2.260, a gap of 0.103. Learned positions add 16,384 parameters.	The fixed code gives useful position information from the first step, while a learned table has to learn it. The gap might shrink with longer training, which we did not test.

All ablations use one seed and 800 steps, so these results describe the early-training regime. The full-budget version is `python -m experiments.ablation --steps 3000 --batch-size 64`, which is in the Colab notebook.

6.3 Sampling strategy

We compared decoding strategies on five prompts, 400 characters each (`outputs/logs/generation_strategy.md`). A "valid word" is one that appears in the training corpus.

Strategy	Valid-word rate	Distinct word bigrams	Repeated 4-gram share
Greedy (T = 0)	0.992	0.117	0.830
T = 0.5	0.958	0.878	0.221
T = 0.8, top-k 40	0.893	0.975	0.120
T = 0.8, top-p 0.9	0.941	0.938	0.153
T = 1.2	0.718	0.989	0.052

Lower temperature gives more valid words and more repetition. Higher temperature gives more variety and more invented words. No setting is best on all three measures. For this model, T = 0.8 with top-p 0.9 is a reasonable compromise.

7. Failure Analysis

Greedy decoding loops. With greedy decoding, the output for the prompt `ROMEO:` repeats the same short phrases. 83% of its character 4-grams are repeats, and only 11.7% of its word bigrams are distinct. With temperature 0.8 and top-k 40, these figures are 12% and 97.5%. Our likely explanation is that argmax always picks the most probable next character. Once a short cycle appears, the model tends to continue it, and a 128-character window gives it no way to break out. Sampling breaks the cycle. The valid-word rate of 99.2% shows that the loop consists of real words, so the text is fluent but empty.

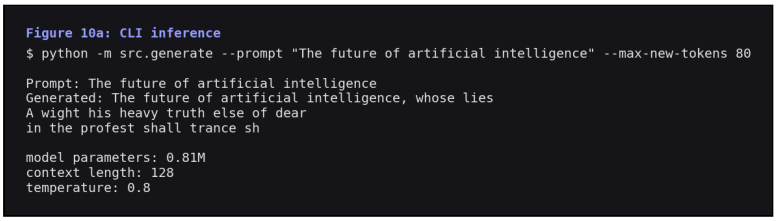
Invented words and no long-range coherence. At temperature 0.8, about 11% of generated words do not appear in the corpus, and at temperature 1.2 the figure is 28%. Scenes have no thread. We think a character-level model has to spell every word, so one wrong character gives a non-word. At 2.37 bits per character the model is still uncertain about many characters, and a 128-character window covers only about 20 words. Larger models, subword tokenisation and longer context are the obvious remedies. We did not test them.

A leaky objective looks good. The bidirectional model reached a validation loss of 0.678, which looks excellent. Under the causal mask used for generation, its loss is 5.69 (Section 2.6). Low training loss means nothing unless the architecture blocks access to the target.

Limits of our evidence. The ablations use one extra seed for the noise estimate and a reduced budget. The learning rate was not tuned. The attention analysis is descriptive. The validation split is contiguous, so it measures generalisation to different plays.

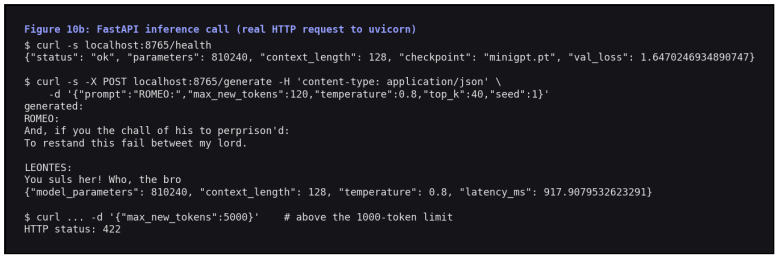
8. Engineering Packaging and Deployment

Command-line interface. `python -m src.generate --prompt "The future of artificial intelligence" --max-new-tokens 80` prints the prompt, the generated text, the parameter count, the context length and the temperature.



REST API. `uvicorn src.api:app` serves four routes: `GET /` (service information), `GET /health` (status, parameter count, context length and validation loss), `POST /generate` (generation with seed, temperature, top-k,

top-p and maximum length) and `GET /docs` (interactive documentation). Requests are validated, so a `max_new_tokens` value above 1,000 returns HTTP 422.



The API is deployed on Render at https://minigpt-3s94.onrender.com, and the interactive documentation is at `/docs`. The repository contains `render.yaml`, a Render blueprint that installs CPU-only PyTorch, and a `Dockerfile` for other hosts. The model is small, but the PyTorch runtime is not, so the free Render tier may run short of memory.

For reproducibility, every hyperparameter is in `configs/tiny_gpt.yaml` and can be overridden with `--set key=value`. Seeds are fixed, the dataset downloads itself, and the trained checkpoint (3 MB) is committed, so inference works without retraining. No API keys or other secrets are used. The Colab notebook (`notebooks/colab_minigpt.ipynb`) runs the full pipeline on a GPU, and Appendix A lists the commands for every figure and table.

9. Conclusion

We built a 0.81 M-parameter, 4-layer GPT in PyTorch, with every Transformer component written by us, and trained it from random initialisation for 38 minutes on a CPU. It reaches 2.37 bits per character on held-out text. It learns the format, vocabulary and spelling patterns of Shakespeare, but not its meaning, and it invents words.

Our work supports four points. First, the implementation behaves as derived: the hand calculation, the unit tests and the trained-model check agree, and future positions receive exactly zero weight. Second, causality rather than capacity is what makes next-token training honest, as the bidirectional experiment shows. Third, depth and fixed sinusoidal positions help at this scale, while head count does not measurably matter. Fourth, decoding strategy matters as much as the weights, since greedy decoding loops and sampling gives varied but sometimes invented text.

The main limits are the model's scale and the strength of the evidence, which rests on single seeds, a reduced ablation budget, and no learning-rate sweep. The next steps would be a learning-rate sweep, several seeds, a width-matched depth control, and a subword tokenizer.

References

1. Vaswani, A., et al. (2017). Attention Is All You Need.
2. Radford, A., et al. (2019). Language Models are Unsupervised Multitask Learners (GPT-2).
3. Devlin, J., et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
4. Karpathy, A. char-rnn and the Tiny Shakespeare dataset (Hugging Face, karpathy/tiny_shakespeare).

The model code is our own. PyTorch provides tensors, autograd, `nn.Linear`, `nn.LayerNorm` and AdamW. `F.scaled_dot_product_attention` is used only inside the tests, as a reference. No pretrained weights or pretrained model classes are used.

Appendix A: Reproduction commands

```
pip install -r requirements.txt
python -m src.dataset --download
python -m pytest -q                                                  # 36 tests

python -m src.train --run-name overfit --overfit-batch \
       --set train.max_steps=300 --set train.batch_size=16       # overfit gate
python -m src.train --run-name minigpt                               # final training run

python -m src.evaluate                                               # loss, perplexity,
samples
python -m src.generate --prompt "ROMEO:" --max-new-tokens 200 --temperature 0.8

python -m experiments.hand_calculation                               # Figures 1, 2
python -m experiments.make_evidence                                  # Figures 3, 4, 5, B
python -m experiments.plot_training                                  # Figure 6
python -m experiments.attention_visualization                        # Figure 7
python -m experiments.ablation                                       # Figure 9
python -m experiments.mlm_vs_clm                                     # Figures C, D
python -m experiments.generation_strategy                            # Section 6.3
python -m experiments.seq_vs_parallel                                # Figure A

uvicorn src.api:app --host 0.0.0.0 --port 8000                       # API
```